



Pointers & Dynamic Memory

Dr. Obed Noé Sámano Abonce

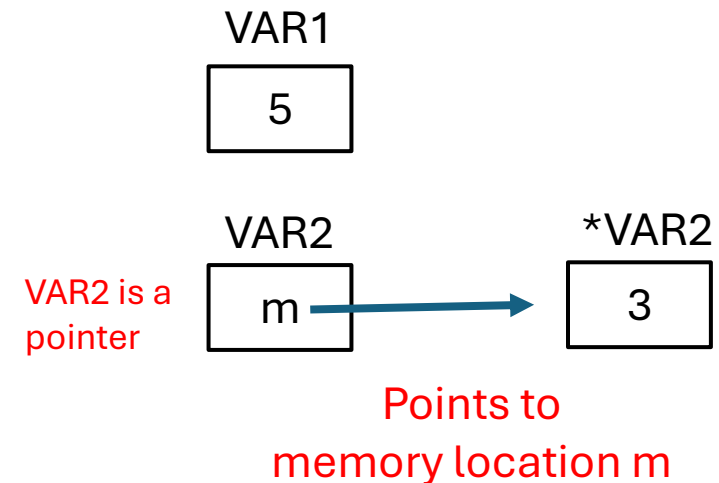
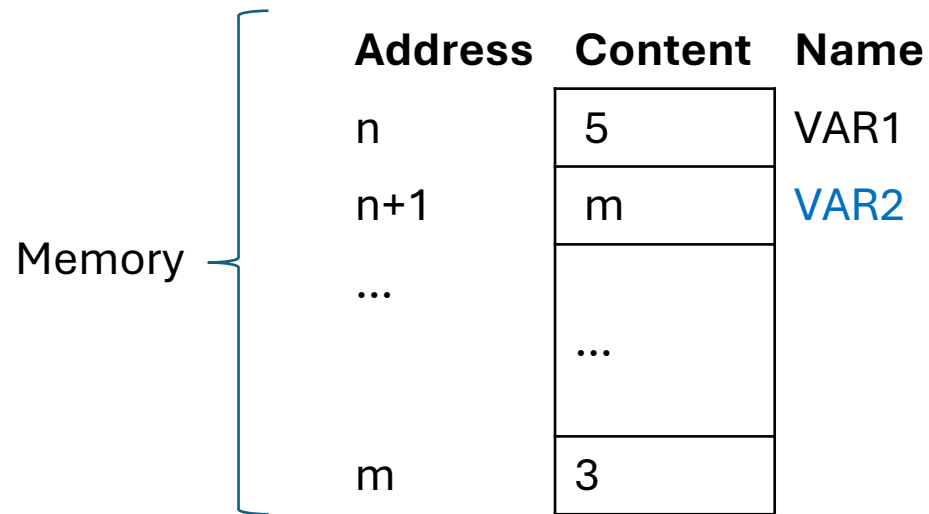


Pointers

What is a pointer?

Pointer

- A pointer holds a memory address where a given element will be stored.
- A pointer is an indirect reference to an element



Pointers

Declaration

```
type *pointerName;
```

Pointer to an int



```
int *p;
```

```
char c, *s;
```



Normal char



Pointer to a char

Pointers

Operations

- Indirection operator [*****]

*****pointerName

← Refers to the space pointed to by the pointer variable...

- Address operator [**&**]

&variable

← Produces the address of the variable.
(The variable can be of any type)

Modularity

“Letting a function reach the caller's variables”

```
void swap(int *a, int *b) { ... }

int main() {
    int z = 1, w = 2;
    swap(&z, &w);    // main es el CALLER
}
```

// swap is the *callee*

Dynamic Memory

“Asking for memory while the program runs”

```
int data[200];           // 200. Always. Decided at compile time.

int *data = new int[n];  // n. Decided while the program runs.
```

Pointers

Application example: Modularity

```
// A - by value
void swap(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

swap(z, w);
```

*It swapped its **own**
two copies —
perfectly. Then **threw**
them away.*

```
// B - by pointer (C)
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

swap(&z, &w);
```

It got the **addresses**
instead of the values
— **so it reached z**
and w themselves.

```
// C - by reference (C++)
void swap(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}

swap(z, w);
```

No copies, no
addresses. **a is**
another name for z.

Pointers Practice

Dynamic Memory

Automatic vs Dynamic Memory

	Automatic	Dynamic
Lives in	Stack	Heap
Created by	The compiler	You – using new
Freed by	The compiler	You – using delete
Lifetime	The enclosing block	Until you delete it
Size	Fixed at compile time	Chosen at run time

Dynamic Memory

Operators new and delete

- The **new** operator
 - Reserves memory and returns its **address**. The pointer stores that address and now points to the new space. If there is no memory available, new **throws** a `std::bad_alloc` exception.
 - Example: `pointer = new dataType;`
- The **delete** operator
 - Releases the memory so the system can reuse it. The pointer is left holding an address it is no longer allowed to use.
 - Example: `delete pointer;`

Dynamic Memory

Example

```
int a = 5, *p;  
p = new int;  
*p = a;  
a = *p + 2;  
cout << a << " " << *p;  
delete p;
```

What's the output?

Dynamic Memory

Example

```
int a = 5, *p;
```

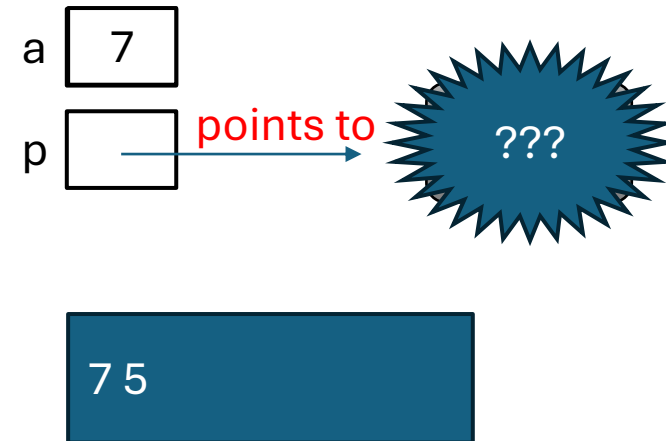
```
p = new int;
```

```
*p = a;
```

```
a = *p + 2;
```

```
cout << a << " " << *p;
```

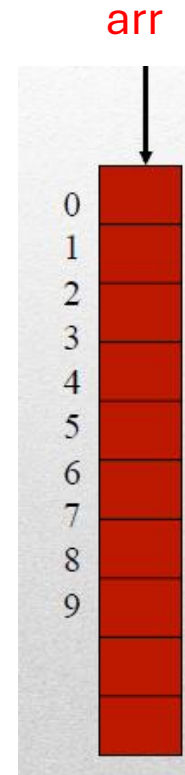
```
delete p;
```



Dynamic Memory

Arrays and pointers

- The name of an array is itself a pointer to the array's 1st element
- Arrays can be handled with pointers:
 - `arr[j]` is equivalent to `*(arr+j)`
 - `&arr[j]` is equivalent to `(arr+j)`
- `arr[0]` is the same as `*arr`
- `&arr[0]` is the same as `arr`



Dynamic Memory

Static Array vs Dynamic Array

Static Array

```
// static – size fixed at compile time
int data[200];

for (j = 0; j < 200; j++)
    cin >> data[j];
```

Dynamic Array

```
// dynamic – size chosen at run time
int *data, n;
cin >> n;

data = new int[n];

for (j = 0; j < n; j++)
    cin >> data[j];

delete[] data;
```

200 is decided when you compile. n is decided while the program runs.
new[] pairs with delete[] — never with plain delete.

Dynamic Memory

Dynamic Objects

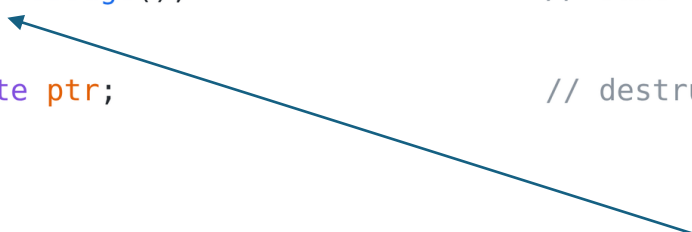
```
class Example { ... };

Example obj;           // on the stack
Example *ptr;          // a pointer to an Example

ptr = new Example;     // default constructor
// or
ptr = new Example(x, y); // with arguments

obj.message();         // stack object: the dot
(*ptr).message();      // dereference, then the dot
ptr->message();         // same thing – and what everyone writes

delete ptr;            // destructor runs, then the memory is freed
```



“->” Operator for reaching members of a pointed-to object.

Dynamic Memory

Objects with dynamic attributes

- The object asks for the **memory** it needs **while the program runs**.
- The **constructor** does the **new**. The **destructor** does the matching **delete**.
- The object cleans up after itself.

```
class Example {  
public:  
    Example() { attr = new int; }           // constructor: the new  
    ~Example() { delete attr; }             // destructor: the delete  
  
private:  
    int *attr;                             // a dynamic attribute  
};
```

Dynamic Memory Practice

References



M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed. Boston, MA, USA: Pearson, 2014.

V. M. de la Cueva Hernández, L. H. González Guerra, y E. G. Salinas Gurrión, *Estructuras de datos y algoritmos fundamentales*. Monterrey, México: Editorial Digital del Tecnológico de Monterrey, 2020.